

COLLEGE MINI PROJECT DOCUMENTATION

Personal Energy Waste Simulator



A software-only household electricity analysis and
potential energy waste simulation system

Java 21 Console Application and Local Web Application

Prepared for academic demonstration
September 2026

Table of Contents

1. Abstract
2. Introduction
3. Requirement Specification
4. Architectural Structure & Design
5. Module Description
6. Implementation
7. Source Code
8. System Screenshots
9. Conclusion
10. Future Enhancement

1. Abstract

Personal Energy Waste Simulator is a beginner-friendly software application that helps households understand electricity consumption and identify potentially avoidable usage. Users enter appliance details such as power rating, number of units, operating hours, and days used per month. The system calculates daily, monthly, and yearly energy consumption and estimates the corresponding electricity cost using a configurable tariff.

The central feature is a transparent potential-waste analysis. Users provide necessary operating hours, and the system estimates additional usage as potential avoidable energy rather than claiming that all additional usage is definitely waste. A what-if simulator compares current and optimized routines, showing energy reduction and possible financial savings. The project is implemented using Core Java and a local browser interface, without hardware, databases, APIs, or artificial intelligence.

2. Introduction

2.1 Background

Electricity bills usually show total usage but do not explain which appliances contribute most or which habits could be reduced. A simple analytical tool can make this information easier to understand and support practical energy-conscious decisions.

2.2 Problem Statement

Common users need a clear way to record household appliances, estimate energy and cost, compare appliances, identify possible avoidable usage, and test improvements without special hardware or technical services.

2.3 Objectives

- Calculate appliance and household energy consumption.
- Estimate daily, monthly, and yearly electricity cost.
- Identify potential avoidable usage using necessary operating hours.
- Simulate optimized usage and calculate possible savings.
- Provide understandable comparisons, scoring, recommendations, and reports.
- Offer both a console application and an easy-to-run local website.

2.4 Scope

The project is intended for education and personal estimation. It does not connect to smart meters, utility providers, IoT devices, or external services. Results depend on the accuracy of user-entered appliance and tariff values.

3. Requirement Specification

3.1 Functional Requirements

ID	Requirement	Implementation
FR-01	Add, edit, remove, and view appliances.	Appliance form and Java <code>ArrayList</code> .
FR-02	Store name, watts, units, hours/day, days/month, and necessary hours.	<code>Appliance</code> model and browser local storage.
FR-03	Calculate daily, monthly, and yearly kWh.	<code>EnergyCalculator</code> and browser calculation functions.
FR-04	Calculate cost with configurable tariff.	Tariff input in console and website.
FR-05	Estimate potential wasted energy and cost.	<code>WasteAnalyzer</code> and Potential Waste view.
FR-06	Compare current and optimized scenarios.	<code>SimulationService</code> and What-if Lab.
FR-07	Rank appliances by monthly consumption.	Sorted comparison table and chart.
FR-08	Generate rule-based recommendations and report.	<code>RecommendationService</code> and report view.
FR-09	Save and load appliance information.	CSV persistence in Java; local storage in website.

3.2 Non-Functional Requirements

- Use Java 21 and Core Java for the main implementation.
- Keep the project understandable for college-level students.
- Handle invalid inputs without crashing.
- Avoid databases, frameworks, APIs, AI/ML, and hardware.
- Provide a responsive, usable local website.

3.3 Validation Rules

Name cannot be empty; watts, units, and tariff must be positive; hours/day must be between 0 and 24; days/month must be between 1 and 31; necessary hours cannot exceed actual hours; division by zero is handled safely.

4. Architectural Structure & Design

The project uses a simple layered design. The model layer represents data, the service layer contains calculations and business rules, the utility layer handles validation and persistence, and the user-interface layer handles console or browser interaction.

Architecture flow:

User Input → Main.java / web interface → Model Objects → Calculation and Analysis Services → Results, Recommendations, and Reports

Layer	Responsibility
Model	Encapsulates appliance, usage, and simulation result data.
Service	Calculates energy, waste, savings, recommendations, and reports.
Utility	Validates values and reads/writes simple CSV data.
UI	Provides the Java menu and responsive browser dashboard.

4.1 Design Principles

- **Encapsulation:** appliance fields are private and accessed through methods.
- **Separation of responsibility:** calculation and reporting logic is kept outside the UI.
- **Composition:** report and recommendation services use calculator and waste analyzer objects.
- **Transparency:** formulas and waste thresholds are shown in the documentation and interface.

5. Module Description

Module	Description
Appliance Management	Creates and maintains appliance records with usage values.
Energy Calculation	Applies the watts, units, hours, and days formula for daily, monthly, and yearly totals.
Cost Calculation	Multiplies energy by the user-configured tariff in Rs./kWh.
Waste Analysis	Compares actual and necessary hours to estimate potential avoidable usage.
What-if Simulation	Creates an optimized scenario and calculates energy, percentage, and money saved.
Comparison	Sorts appliances by monthly consumption and visualizes the highest loads.

Recommendations	Uses simple conditions for high consumption, high contribution, and potential waste.
Report	Combines totals, costs, waste, highest appliances, savings, and recommendations.
Persistence	Saves Java records in <code>appliances.csv</code> ; website records use browser local storage.

6. Implementation

6.1 Main Formulae

```

Daily kWh = (watts × units × hours per day) / 1000
Monthly kWh = daily kWh × days used per month
Yearly kWh = monthly kWh × 12
Potential wasted kWh = (actual hours - necessary hours) × watts × units × days / 1000
Cost = kWh × tariff

```

6.2 Waste Score

Potential waste percentage	Score
0 to below 5%	Very Low
5 to below 15%	Low
15 to below 30%	Moderate
30 to below 50%	High
50% and above	Very High

This is a project-defined indicator and is not an official energy-efficiency standard.

6.3 Technologies

- Java 21, classes, constructors, encapsulation, methods, ArrayList, exception handling, and file I/O.
- HTML, CSS, and JavaScript for the browser interface.
- Node.js built-in HTTP module for local serving.
- Browser local storage for website persistence.

6.4 Running the Project

For the website, Windows users can double-click `Start Personal Energy Simulator.bat`. Alternatively run `node server.js` and open `http://localhost:3000`. For Java, compile with `javac -d out src\\Main.java src\\model\\*.java src\\service\\*.java src\\util\\*.java` and run with `java -cp out Main`.

7. Source Code

The complete source code is included in the project repository. The main source files are listed below.

Path	Purpose
src/Main.java	Console menu, input collection, and application flow.
src/model/Appliance.java	Appliance data and encapsulated properties.
src/service/EnergyCalculator.java	Energy and cost-related calculations.
src/service/WasteAnalyzer.java	Potential waste, percentage, and score logic.
src/service/SimulationService.java	Current versus optimized scenario comparison.
src/service/RecommendationService.java	Rule-based recommendations.
src/service/ReportService.java	Final text report generation.
src/util/InputValidator.java	Input range and value validation.
src/util/FileManager.java	CSV save and load operations.
web/index.html, web/styles.css, web/app.js	Website structure, visual design, and browser behavior.
server.js	Dependency-free local static web server.

Representative Java calculation code:

```
public double monthlyKwh(Appliance appliance) {
    return dailyKwh(appliance) * appliance.getDaysPerMonth();
}

public double wastedDailyKwh(Appliance appliance) {
    double hours = Math.max(0,
        appliance.getHoursPerDay()
        - appliance.getNecessaryHoursPerDay());
    return hours * appliance.getPowerWatts()
        * appliance.getUnits() / 1000;
}
```

8. System Screenshots

The implemented website provides a responsive dashboard with overview metrics, monthly load visualization, waste score, appliance management, potential waste ranking, what-if simulation, and a printable report.

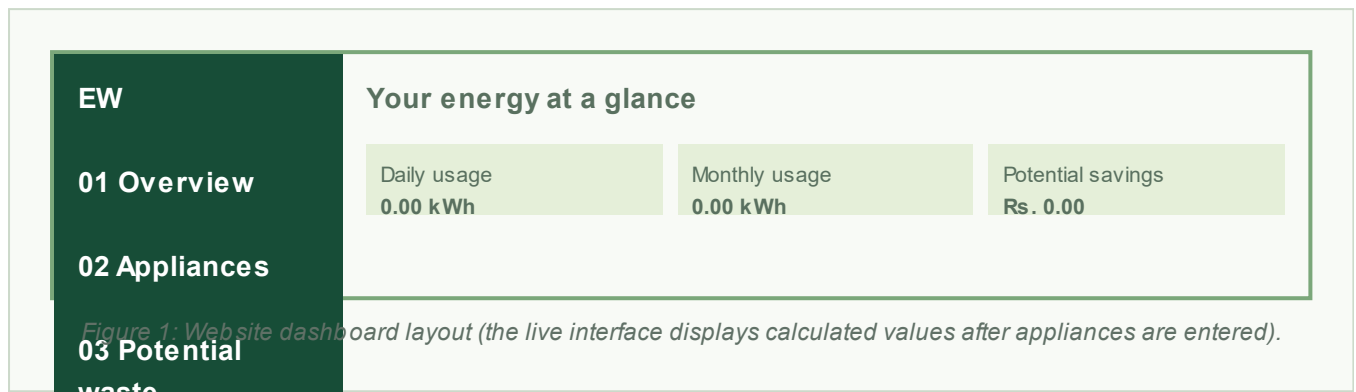


Figure 1: Website dashboard layout (the live interface displays calculated values after appliances are entered).

For actual screenshots, open the local website with `node server.js`, visit `http://localhost:3000`, load the example household, and capture the Overview, Potential Waste, What-if Lab, and Full Report views.

9. Conclusion

Personal Energy Waste Simulator meets its objective of making household electricity usage easier to understand. It accepts simple appliance information, applies clear formulas, estimates energy and cost, highlights possible avoidable usage, and demonstrates how small usage changes can create measurable savings. The project also demonstrates practical object-oriented design through separate models, services, utilities, and interfaces.

The system is intentionally transparent: potential waste is an estimate based on user assumptions, and the score is a project-defined indicator. This makes the application suitable for a college demonstration while avoiding unsupported claims about real-world energy efficiency.

10. Future Enhancement

- Add tiered electricity tariffs and different tariff plans.
- Support multiple household profiles and user accounts.
- Add a backend database for shared or cloud-based data.
- Provide PDF export directly from the website.
- Add seasonal, weekday, and weekend usage patterns.
- Include charts for historical consumption and savings.
- Add multilingual support for wider accessibility.
- Connect to official utility data only if a secure and approved API becomes available.